

Motion in Images

Optical flow, two-frame tracking, and four-view planar reconstruction

CSc 8830 Computer Vision | Module 6 | Sirichandana Bikkasani

Project question

How can image motion be estimated between frames, checked at the pixel level, and combined across multiple viewpoints to recover scene structure? This project implements dense optical flow, local feature tracking, subpixel interpolation, and a calibrated planar multi-view reconstruction.

Assignment goals and current evidence

Requirement	Evidence in this project
Two motion videos	35.83 s unpacking/assembly clip and 45.03 s continuous road-traffic clip. Both are over 30 seconds, show visible motion, and fit the 20 MB app upload limit as prepared MP4 files.
Dense optical flow	Farneback flow was computed over the first 30 s of each clip and saved as a 30 s side-by-side MP4.
Two-frame tracking and pixel validation	80 features per clip were tracked; two approximate Frame B readings are compared with the predictions using endpoint error.
Four-view reconstruction	Completed using four included checkerboard images and prior phone calibration.

Input note: Both clips are continuous recordings longer than 30 seconds. Each dense-flow output covers the first 30 seconds. The capture rate includes repeated frames, so the tracking check advances to the next distinct image when necessary.

1. Optical flow and point tracking

Brightness constancy assumes that a moving image point keeps approximately the same intensity over a small time interval. A first-order Taylor expansion produces the optical-flow constraint:

$$I(x+u*dt, y+v*dt, t+dt) = I(x, y, t)$$
$$I_x*u + I_y*v + I_t = 0$$

This is one equation with two unknown velocity components, so a single pixel is insufficient. Lucas-Kanade assumes a small neighborhood shares one motion vector. For pixels i in that window, the equations form $A d = b$, with $d = [u, v]$ and $b = -I_t$. The least-squares estimate minimizes squared error, and when $\text{transpose}(A)*A$ is invertible, it is:

$$d = \text{inverse}(\text{transpose}(A)*A) * \text{transpose}(A) * b$$

The app detects Shi-Tomasi corners in frame t and uses pyramidal Lucas-Kanade to estimate their positions in the adjacent frame. A numbered side-by-side visualization and CSV expose (x_0, y_0) , (x_1, y_1) , displacement, and per-axis motion. The manual check compares a visually read location with the estimate using endpoint error = $\sqrt{\text{square}(x_{\text{manual}}-x_1) + \text{square}(y_{\text{manual}}-y_1)}$.

Subpixel interpolation

A tracked location is generally not at integer pixel coordinates. For (x_0+a, y_0+b) , bilinear interpolation blends the four surrounding grayscale samples:

$$I = (1-a)(1-b)I_{00} + a(1-b)I_{10} + (1-a)bI_{01} + a*b*I_{11}$$

Implementation and interpretation

The dense field uses OpenCV Farneback flow and maps direction to hue and magnitude to brightness. The video is sampled at up to 10 frames per second and the app displays the original beside the flow field. Flow is an apparent image-motion estimate; camera movement, illumination change, blur, occlusion, and texture ambiguity can make it differ from physical object motion.

2. Four-view planar reconstruction

The planar target is a 9 by 6 internal-corner checkerboard with 25 mm square spacing. Four images from the earlier phone-calibration exercise are included with this project. The previously estimated intrinsic matrix is $K = \begin{bmatrix} 3095.067 & 0 & 932.214 \\ 0 & 3094.479 & 2024.674 \\ 0 & 0 & 1 \end{bmatrix}$ for 1884 by 4080 images. The distortion coefficients are $k1 = 0.069978$, $k2 = 0.921953$, $p1 = -0.004310$, $p2 = -0.002685$, and $k3 = -4.709119$.

For each view, the checkerboard grid provides 3D reference points $X_i = (25c, 25r, 0)$ mm and detected image corners x_i . OpenCV solvePnP estimates rotation R_i and translation t_i under the calibrated camera model. The camera center in board coordinates is $C = -\text{transpose}(R)*t$. Adjacent views are undistorted, then corresponding corner rays are triangulated by minimizing their reprojection mismatch. Projection follows:

x_{image} is proportional to $K [R \mid t] X_{\text{world}}$

The known grid makes this a calibrated planar reconstruction with known geometry and metric scale. It is not unconstrained SfM of an unknown scene: pose estimation uses the known board dimensions. This controlled setup is appropriate for checking the multi-view geometry pipeline and its error.

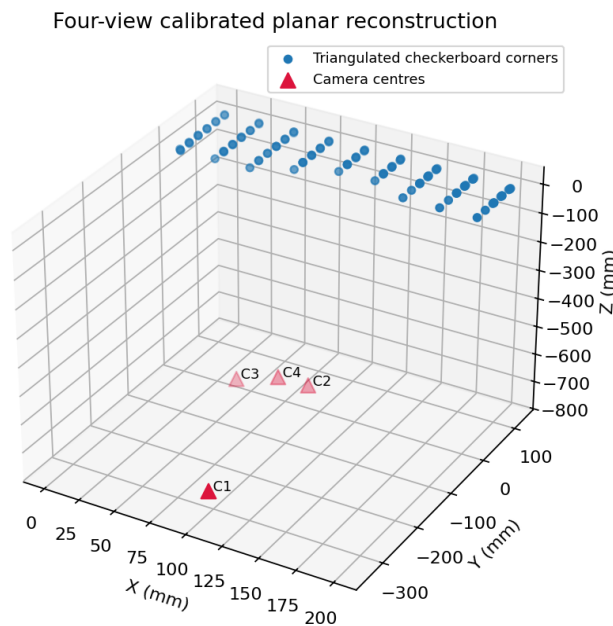


Figure 1. Triangulated checkerboard corners and four estimated camera centers; coordinates are in the checkerboard coordinate frame.

Estimated camera centers

View / image	X (mm)	Y (mm)	Z (mm)
1 / view_01_calib_05.jpeg	107.4	-340.2	-680.3
2 / view_02_calib_06.jpeg	91.5	0.2	-721.2
3 / view_03_calib_09.jpeg	43.7	-17.1	-749.9
4 / view_04_calib_11.jpeg	74.3	-18.0	-696.0

3. Results and validation

Measure	Observed result
Calibration reprojection RMS	0.722 px (prior calibration result)
Detected / triangulated points	54 corners per view; 114 triangulated points across adjacent pairs
Points by adjacent pair	1-2: 34; 2-3: 43; 3-4: 37
Known board extent	200 x 125 mm
Reconstructed extent	200.64 x 125.89 mm
Extent difference	+0.64 x +0.89 mm
Depth spread of triangulated corners	10.62 mm (ideal planar target: 0 mm)

Interpretation

The reconstructed width and height are close to the known checkerboard extent, with about 0.64 mm and 0.89 mm of extent difference. The 10.62 mm depth spread shows noticeable non-planarity in the triangulated estimates. Possible contributors include calibration residual, corner localization, small camera-baseline changes, and planar-scene geometry. The point-cloud extents can be checked directly: X extent = $200.365 - (-0.274) = 200.639$ mm; Y extent = $125.739 - (-0.153) = 125.892$ mm. Against the known 200 by 125 mm grid, the differences are +0.639 mm and +0.892 mm. The reported boundary is the point-cloud extent, not a direct hidden-object boundary detector.

4. Motion video results and tracking check

Input	Duration; analyzed	Flow median*	LK tracks; median / max motion	Frame B: predicted / visual reference (px)	Endpoint error
Video 1: Person unpacking and assembling a box item	35.83 s; first 30 s	0.061	80; 0.000 / 3.93	Track 53: (301.91,291.52) / approx. (302,292)	0.48 px
Video 2: Continuous road traffic	45.03 s; first 30 s	0.009	80; 0.028 / 9.60	Track 6: (361.35,304.74) / approx. (361,305)	0.43 px

*Median over all pixels; stationary background pixels are included.

Pixel-coordinate check

Track 53 in Video 1 moves from (298.00, 292.00) px by (3.91, -0.48) px to (301.91, 291.52) px. Track 6 in Video 2 moves from (353.00, 300.00) px by (8.35, 4.74) px to (361.35, 304.74) px. These are the Lucas-Kanade pixel-coordinate updates for the selected frame pairs.

The Frame B reference locations were read approximately from enlarged images, not obtained from independent ground truth. Video 1 uses track 53 at 12.000 to 12.033 seconds: predicted (301.91, 291.52) px versus visual reference about (302, 292) px. Endpoint error = $\sqrt{(302-301.91)^2 + (292-291.52)^2} = 0.48$ px. Video 2 uses track 6 at 15.000 to 15.033 seconds: predicted (361.35, 304.74) px versus visual reference about (361, 305) px, giving 0.43 px endpoint error. Bilinear grayscale values at the tracked subpixel points are 164.0 and 136.1 on a 0-255 scale. These approximate visual checks should be described as estimates, not exact ground truth.

Bilinear interpolation example

For Video 2, Track 1 ends at $x=470.171875$ and $y=108.110947$ px. Thus $\alpha=0.171875$, $\beta=0.110947$, and the four grayscale values are $I_{00}=131$, $I_{10}=147$, $I_{01}=153$, $I_{11}=162$. Substitution into the bilinear equation gives $(1-0.171875)(1-0.110947)(131) + 0.171875(1-0.110947)(147) + (1-0.171875)(0.110947)(153) + (0.171875)(0.110947)(162) = 136.06$. The code returns 136.06 on a 0 to 255 grayscale scale.

Interpretation

The person clip shows localized hand and body motion against a mostly stationary room. The traffic clip shows moving vehicles against a largely stationary camera view. Dense-flow median is measured over the full image, so a large static background keeps it low even when a smaller region is moving. The original 60 fps captures contain repeated images. The app inputs are resized MP4 copies at 30 fps, with repeated images removed. Lucas-Kanade can still fail with blur, occlusion, low texture, or large motion. The four-view reconstruction uses a calibrated checkerboard with known geometry, rather than unconstrained reconstruction of an unknown scene.

Reproducibility and references

The Streamlit application, source, four checkerboard images, calibration JSON, 3D point CSV, camera-center CSV, and reconstruction plot are in the Module6_OpticalFlow_SfM folder of the course GitHub repository. Use the README for setup. The analysis output can be recreated by running `python -m src.create_sfm_outputs` from the project folder.

Camera setup and submitted input images

The included camera positions are estimated relative to the checkerboard plane from calibrated phone images. The camera intrinsics are reused from the Module 2 calibration; the included JSON records the image size, distortion coefficients, board grid, and calibration RMS. Four original viewpoint images are included under `data/sfm_views/`.

References

1. Szeliski, R. *Computer Vision: Algorithms and Applications*, 2nd ed. Optical flow, feature tracking, and multi-view geometry chapters. <https://szeliski.org/Book/>
2. OpenCV documentation. Optical Flow: Farneback and Pyramidal Lucas-Kanade methods. https://docs.opencv.org/4.x/d4/dee/tutorial_optical_flow.html
3. OpenCV documentation. Camera calibration, pose estimation, and 3D reconstruction functions: `findChessboardCornersSB`, `solvePnP`, `undistortPoints`, and `triangulatePoints`. <https://docs.opencv.org/4.x/>
4. Course reference playlist: Optical Flow | Structure from Motion | Object Tracking. <https://www.youtube.com/playlist?list=PL2zRqk16wsdoYzrWStffqBAoUY8XdvatV>
5. Previous camera calibration measurements and images produced for CSc 8830 Module 2; calibration reprojection RMS 0.722 px.

Repository: https://github.com/siri423/CSc8830-Computer-Vision/tree/main/Module6_OpticalFlow_SfM